

Mask2Former : Transformateur de Masques avec Attention Masquée pour la Segmentation Universelle d'Images

Application à la Segmentation d'Instances de Bâtiments Marocains à partir d'Imagerie
Satellite

Analyse Mathématique et Implémentation

19 août 2025

Plan de la Présentation

- 1 Contexte et Problématique
- 2 Présentation du Modèle Mask2Former
- 3 Du Backbone au Décodeur de Pixels
- 4 Le Transformer Decoder comme Étape Finale
- 5 Fonctions de Perte (Loss Functions)
- 6 Implémentation et Ajustement Fin (Fine-Tuning)
- 7 Résultats et Analyse

Contexte du Problème : Segmentation de Bâtiments Marocains

Objectif

Développer un modèle de segmentation pour identifier et délimiter les bâtiments individuels dans l'imagerie satellite marocaine



Résidentiel

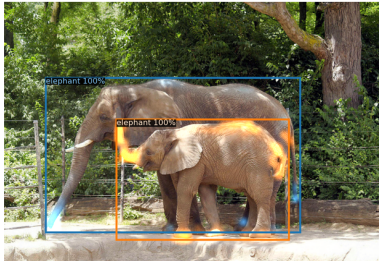


Appartements

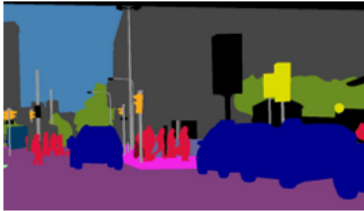


Autres (Offices, écoles...)

Types de segmentation



Segmentation d'instances



Sémentation



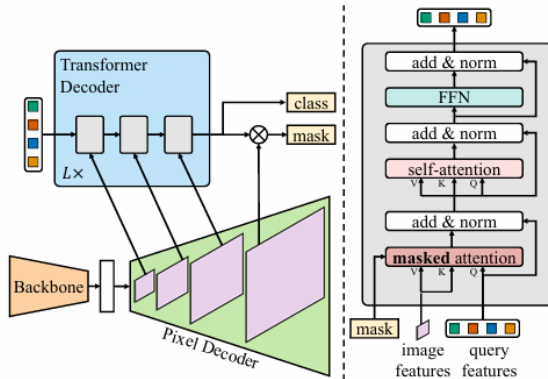
Panoptique

Introduction à Mask2Former

Une architecture unifiée pour la segmentation d'images

- Traditionnellement, chaque tâche nécessite une architecture spécialisée.
- **Mask2Former** propose une solution unifiée pour toutes ces tâches.
- Basé sur un Transformer avec **attention masquée**, il extrait des caractéristiques localisées à partir de régions de masques prédits.
- Cette approche simplifie la recherche et l'implémentation, tout en maintenant des performances de pointe.

Mask2Former : Une architecture globale



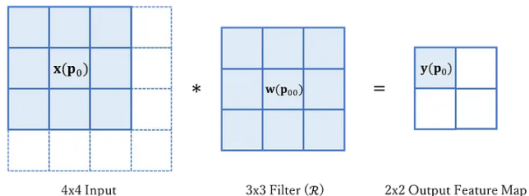
Convolution 2D Classique

- Soit x une carte de caractéristiques d'entrée.
- $R = \{p_n\}_{n=1}^N$: grille régulière (ex. 3×3).
- $w(p_n)$: poids associés à chaque point p_n .

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n)$$

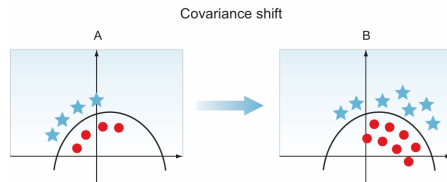
- p_0 : position centrale dans la sortie y .
- $\Rightarrow$ Produit scalaire entre le filtre et sa région réceptrice.

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n)$$



Covariate Shift : Problème de Distribution

- Supposons que l'on entraîne un classifieur uniquement sur des images de chats blancs.
- Lors du test, les chats ont des couleurs variées
⇒ performance faible.
- Pourquoi ? La distribution d'entrée a changé entre l'entraînement et le test.
- Ce phénomène est appelé **Covariate Shift**.
- Ce même problème apparaît entre les couches d'un réseau profond :
 - Les activations changent constamment, perturbant l'apprentissage des couches suivantes.
- **Batch Normalization** est une solution à ce problème.



Graph A : chats blancs (train) — Graph B : chats de différentes couleurs (test)

Batch Normalization : Normaliser les Activations

Stabiliser la distribution des entrées dans chaque couche

Hypothèses :

- Soit un mini-batch de m exemples : $x_1, x_2, \dots, x_m$
- On applique BN **par canal** après la convolution

Étapes de Batch Normalization :

① Calcul des statistiques de mini-batch :

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i \quad ; \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

② Normalisation :

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

③ Rééchelonnage et décalage :

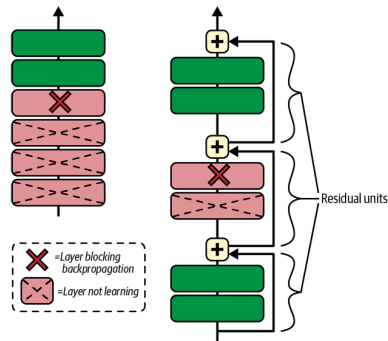
$$y_i = \gamma \cdot \hat{x}_i + \beta \quad \text{avec} \quad \gamma, \beta \text{ apprises pendant l'entraînement}$$

Backbone : ResNet - Résoudre le problème du gradient qui disparaît

Dans les réseaux neuronaux profonds, le **gradient disparaît** lors de la rétropropagation, ce qui rend l'apprentissage des premières couches difficile.

Connexions de saut :

- Création d'un **chemin alternatif** pour faciliter la propagation du gradient.
- Transmission directe de l'information d'une couche à une autre plus profonde.
- Possibilité d'**apprendre la fonction identité** : l'entrée est copiée à la sortie si aucune amélioration n'est possible.
- Garantit que les nouvelles couches ne dégradent pas les performances.



Réseau neuronal profond classique (à gauche) et réseau neuronal profond résiduel (à droite)

Backbone : ResNet - Principe du Bloc Résiduel

Structure :

- Deux chemins :
 - **Principal** : transforme l'entrée x en $\mathcal{F}(x)$
 - **Raccourci** : transmet x directement
- Chemin principal :
 - 1×1 conv $\rightarrow$ BN $\rightarrow$ ReLU
 - 3×3 conv $\rightarrow$ BN $\rightarrow$ ReLU
 - 1×1 conv $\rightarrow$ BN
- Chemin de raccourci :
 - *Identité* si les dimensions sont égales
 - *Projection* ($W_s x$) si les dimensions diffèrent

Formule :

$$H(x) = \mathcal{F}(x) + x \quad \text{ou} \quad \mathcal{F}(x) + W_s x$$

$$\text{Sortie} = \text{ReLU}(H(x))$$

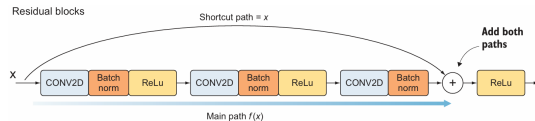
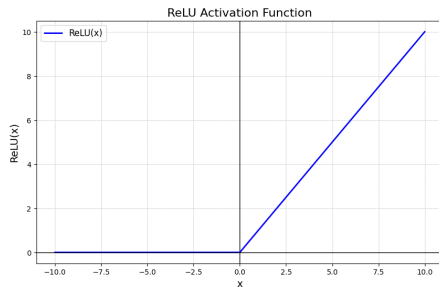


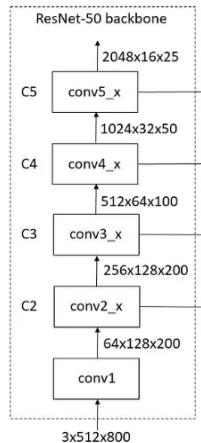
Schéma d'un bloc résiduel avec connexion directe



Extraction des Caractéristiques du Backbone

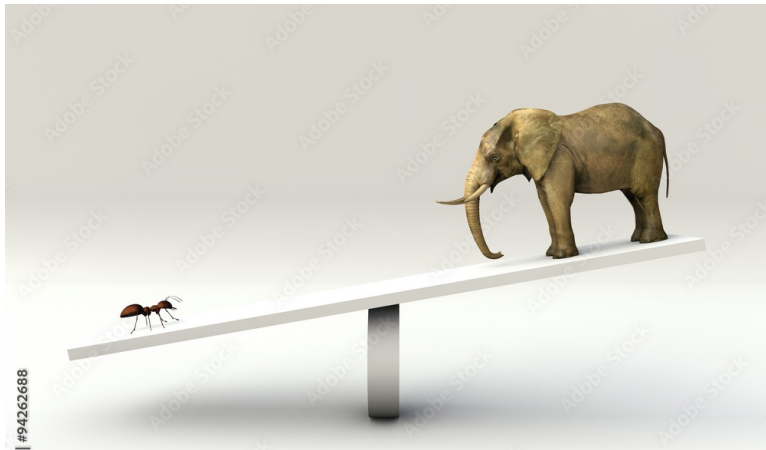
Sortie Multi-Résolution

- **C2** : Stride 4, résolution $\frac{H}{4} \times \frac{W}{4}$
- **C3** : Stride 8, résolution $\frac{H}{8} \times \frac{W}{8}$
- **C4** : Stride 16, résolution $\frac{H}{16} \times \frac{W}{16}$
- **C5** : Stride 32, résolution $\frac{H}{32} \times \frac{W}{32}$



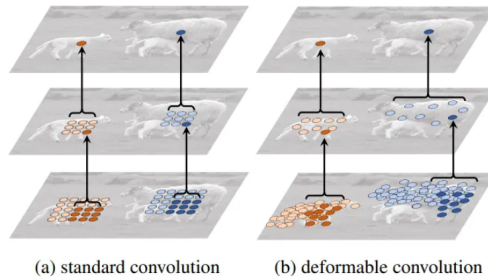
Après ResNet, quelle étape suivante ?

- ResNet nous donne des informations à plusieurs niveaux dans l'image,
- Mais comment utiliser ces informations pour mieux détecter des objets petits ou grands ?
- Comment combiner des détails précis et des informations globales ?



Limite de la Convolution Standard

- Le champ récepteur est **fixe** et **rigide**.
- Ne peut pas s'adapter à des formes ou objets déformés.
- Exemples : objets inclinés, formes non alignées...



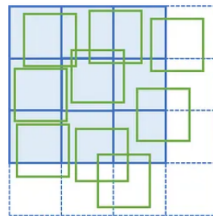
Convolution Déformable : Principe

- On introduit des offsets appris Δp_n :

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n + \Delta p_n)$$

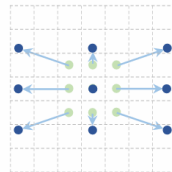
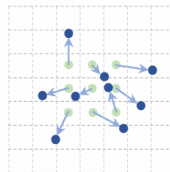
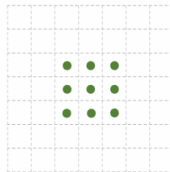
- Δp_n sont des valeurs réelles (fractionnaires).
- La grille R devient **irrégulière**, dynamique.

$$y(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n + \Delta \mathbf{p}_n)$$



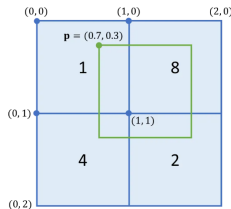
4x4 input with 3x3 filter and offsets applied (green).

- $\Delta p_n \in \mathbb{R}^2 \rightarrow$ déplacement sous-pixel.
- Si Δp_n est fractionnaire, comment peut-on connaître la valeur de $x(p_0 + p_n + \Delta p_n)$?
- On ne peut pas la connaître avec certitude, mais on peut en estimer une valeur très précise en utilisant l'**interpolation bilinéaire**.



Interpolation bilinéaire

- Estime une valeur entre 4 points connus dans un rectangle.
- Utilisée pour approximer $x(p_0 + p_n + \Delta p_n)$ lorsque la position est fractionnaire.
- Utilise un noyau $G(q, p)$ d'interpolation bilinéaire.
- p : position décalée cible.
- q : positions entières voisines dans la carte x .



$$x(p) = \sum_q G(q, p) \cdot x(q)$$

where
$$\begin{cases} G(q, p) = g(q_x, p_x) \cdot g(q_y, p_y) \\ g(a, b) = \max(0, 1 - |a - b|) \end{cases}$$

$$\begin{aligned} x(p) = & \max(0, 1 - |0 - 0.7|) \cdot \max(0, 1 - |0 - 0.3|) \cdot x(0, 0) \\ & + \max(0, 1 - |1 - 0.7|) \cdot \max(0, 1 - |0 - 0.3|) \cdot x(1, 0) \\ & + \max(0, 1 - |0 - 0.7|) \cdot \max(0, 1 - |1 - 0.3|) \cdot x(0, 1) \\ & + \max(0, 1 - |1 - 0.7|) \cdot \max(0, 1 - |1 - 0.3|) \cdot x(1, 1) \end{aligned}$$

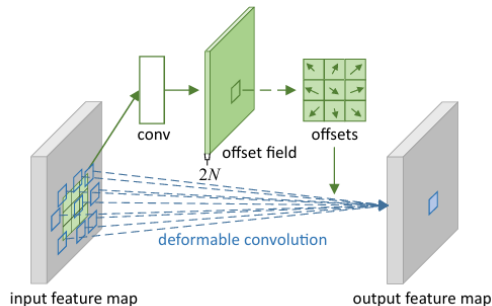
$$\begin{aligned} x(p) = & 0.3 \cdot 0.7 \cdot 1 \\ & + 0.7 \cdot 0.7 \cdot 8 \\ & + 0.3 \cdot 0.3 \cdot 4 \\ & + 0.7 \cdot 0.3 \cdot 2 \\ x(p) = & 4.91 \approx 5 \end{aligned}$$

Formulation Complète : Convolution Déformable

- Expression finale :

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot \sum_q G(q, p_0 + p_n + \Delta p_n) \cdot x(q)$$

- $G(q, \cdot)$: interpolation.
- Δp_n : appris $\rightarrow$ permet d'ajuster la grille au contenu.



- Dans les Transformers, l'attention permet d'agréger dynamiquement des contenus pertinents à partir d'un ensemble de clés.
- Exemple : un mot cible (requête) peut extraire des informations utiles à partir des mots sources (clés) dans une phrase.
- Cette pondération est guidée par la compatibilité entre la requête et chaque clé.

Question : peut-on faire la même chose avec des pixels ou des régions d'une image ?

Pourquoi l'attention dans les images ?

- Dans une image, certaines régions influencent le traitement d'autres régions.
- Exemple : une partie d'un objet dépend du contexte spatial global (forme, symétrie...).
- Idée : permettre à une région (requête) de "regarder" d'autres (clés).

Formulation de l'attention dans une image (1/2)

- On considère une image transformée en carte de caractéristiques : $\mathcal{F} \in \mathbb{R}^{H \times W \times C}$.
 - H : hauteur (nombre de lignes de la grille)
 - W : largeur (nombre de colonnes de la grille)
 - C : nombre de canaux (profondeur de caractéristiques à chaque position)
- Chaque "pixel" dans $\mathcal{F}$ contient un vecteur de caractéristiques de dimension C : il encode l'information visuelle locale.
- On a donc $N = H \times W$ positions distinctes sur lesquelles on va appliquer l'attention.

De la carte de caractéristiques à une séquence de vecteurs

Préparation des données pour le Transformer

Données d'entrée :

$\mathcal{F} \in \mathbb{R}^{C \times H \times W}$ avec C : canaux, $H \times W$: dimensions spatiales

Étapes de transformation :

- 1 **Permutation** des dimensions pour placer la spatialité en premier :

$$\mathcal{F}' \in \mathbb{R}^{H \times W \times C}$$

- 2 **Aplatissement spatial** (concaténation ligne par ligne) :

$$X = \text{reshape}(\mathcal{F}') \in \mathbb{R}^{N \times C}, \quad N = H \times W$$

Chaque vecteur $x_k \in \mathbb{R}^C$ représente un pixel (ou patch) dans la carte.

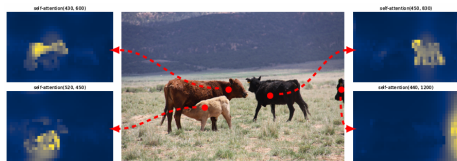
- 3 **But** : convertir une image ou une feature map en une séquence adaptée à l'attention.

Formulation de l'attention dans une image (2/2)

- On transforme la carte $\mathcal{F}$ en une séquence de vecteurs : $\{x_1, x_2, \dots, x_N\}$ où $x_k \in \mathbb{R}^C$.
- Chaque vecteur x_k est enrichi d'un encodage de position (souvent par sommation ou concaténation d'un vecteur de position).
- On choisit une position q dans cette grille — appelée la "requête" (query).
- Cette requête va comparer sa représentation z_q avec toutes les représentations x_k (appelées "Keys") pour décider où "attendre" (Attend).
- Cette interaction est réalisée via un produit scalaire projeté suivi d'une normalisation (softmax) : c'est le poids d'attention.

Interaction entre régions visuelles

- Chaque région de l'image pose une question : "où est l'information utile pour moi?"
- Elle interroge toutes les autres régions.
- Chacune propose une réponse pondérée selon sa pertinence.



Projections linéaires : vues apprises de la carte (1/2)

- Objectif : comparer efficacement les régions de l'image. Mais comment ?
- L'idée : apprendre différentes "vues" ou interprétations des vecteurs x_k et z_q .
- Problème : l'espace d'origine ($\mathbb{R}^C$) peut ne pas être optimal pour les comparaisons.
- Solution : projeter chaque vecteur dans un autre espace où la similarité est plus significative.
- On introduit alors trois transformations linéaires :

$$q = Uz_q, \quad k = Vx_k, \quad v = Wx_k$$

- Les matrices $U, V, W \in \mathbb{R}^{C' \times C}$ sont apprises pendant l'entraînement.
- Ces transformations permettent au modèle de "regarder" les vecteurs selon des critères pertinents.

Projections linéaires : vues apprises de la carte (2/2)

- z_q est la représentation de la région cible (ex : un pixel dans une carte profonde).
- x_k est la représentation d'une région potentiellement informative.
- Pourquoi trois projections différentes ?
 - q (query) : ce que cherche la requête.
 - k (key) : ce que chaque région a à offrir.
 - v (value) : l'information que l'on récupérera si k est jugé pertinent.
- L'idée est donc de transformer les représentations pour qu'elles puissent interagir de façon significative.

Score d'attention entre régions (1/2)

- Comment une région sait-elle où regarder ?
- Une fois q et k obtenus, on calcule leur similarité :

$$s_{qk} = q^\top k = z_q^\top U^\top V x_k$$

- C'est un produit scalaire dans l'espace projeté : plus q et k pointent dans la même direction, plus le score est élevé.
- Ce score exprime à quel point la région x_k est pertinente pour z_q .

Score d'attention entre régions (2/2)

- Pour obtenir des poids interprétables, on normalise tous les scores par rapport à k via softmax :

$$A_{qk} = \frac{\exp(s_{qk})}{\sum_j \exp(s_{qj})}$$

- Ainsi, chaque $A_{qk} \in [0, 1]$ exprime l'importance relative de x_k pour la requête q .
- Ces poids servent ensuite à combiner les valeurs v associées à chaque x_k pour produire la sortie.

Récupération d'information depuis les régions (1/2)

- Maintenant que chaque score A_{qk} représente combien la requête z_q doit écouter la région x_k ...
- Que veut-on récupérer ? De l'information utile.
- Cette information est encodée dans $v = Wx_k$, une version transformée de x_k .
- On effectue donc une somme pondérée de toutes les valeurs :

$$\text{Sortie}(z_q) = \sum_k A_{qk} \cdot v_k = \sum_k A_{qk} \cdot Wx_k$$

- Cela donne une nouvelle représentation pour z_q , enrichie du contexte global.

Récupération d'information depuis les régions (2/2)

- Cette opération est effectuée pour chaque requête z_q de la carte.
- On obtient une nouvelle carte de caractéristiques, avec le même nombre de positions, mais un contenu transformé.
- Intuition : chaque région "voit" ce que les autres ont à offrir, puis synthétise une nouvelle représentation à partir des plus pertinentes.
- Ce mécanisme permet la communication entre régions éloignées de l'image.

Multi-Head Attention : plusieurs regards sur l'image (1/2)

- Question : et si une seule projection U, V, W ne suffisait pas pour capturer tous les types de relations ?
- Idée : créer plusieurs ensembles de projections U_m, V_m, W_m .
- Chaque ensemble m forme ce qu'on appelle une **tête** d'attention.
- Chaque tête produit sa propre sortie :

$$\text{Sortie}_m(z_q) = \sum_k A_{mqk} \cdot W_m x_k$$

- On obtient ainsi M points de vue différents pour chaque requête.

Multi-Head Attention : plusieurs regards sur l'image (2/2)

- On concatène les M sorties (une par tête), puis on applique une projection finale :

$$\text{MHA}(z_q) = W_o \cdot \text{Concat}[\text{Sortie}_1, \dots, \text{Sortie}_M]$$

- W_o ramène la concaténation à la dimension initiale C .
- Intuition : chaque tête capture un type de relation spatiale différente.
- Le modèle peut ainsi apprendre à repérer plusieurs motifs simultanément : proximité locale, symétrie, co-occurrence visuelle, etc.

Résumé : ce que nous avons construit (et pourquoi)

- On part d'une carte de caractéristiques visuelles $\mathcal{F}$.
- Chaque région z_q y devient une requête qui cherche à comprendre son contexte.
- On apprend à projeter les régions (q, k, v) pour mieux comparer et récupérer l'information.
- L'attention pondère l'importance des autres régions pour chaque position.
- Les têtes multiples offrent plusieurs perspectives complémentaires.
- **Résultat** : une nouvelle carte où chaque position intègre de l'information contextuelle globale.

C'est la base attentionnelle de tous les Transformers en vision.

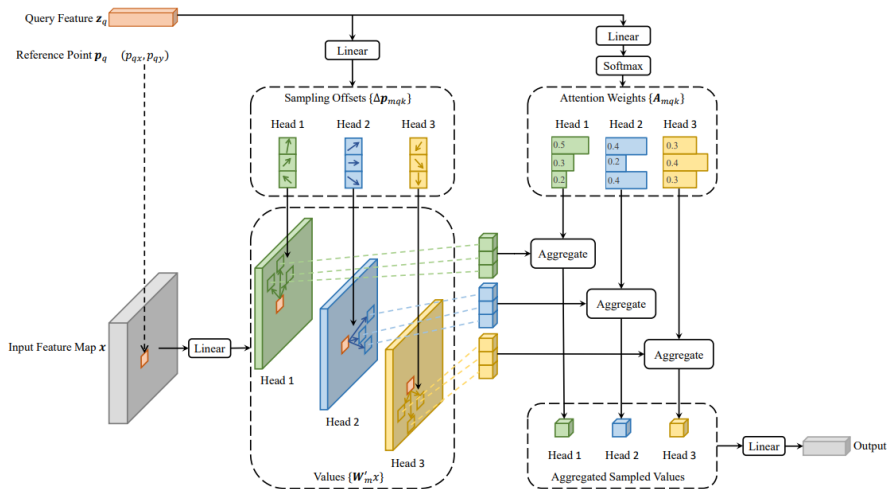
Pourquoi une attention déformable ? (1/2)

- Problème : l'attention standard regarde *toutes* les positions de la carte de caractéristiques.
- Conséquence : coût quadratique en mémoire et calcul si la carte est grande.
- Observation clé : toute région d'une image n'a pas besoin d'observer partout — l'information utile est souvent locale ou semi-locale.
- Idée : s'inspirer des **convolutions déformables**
- Échantillonner seulement quelques points pertinents autour d'une position de référence.

Pourquoi une attention déformable ? (2/2)

- On garde l'idée d'attention : pondérer plusieurs régions.
- Mais on ne regarde plus **toute** la carte — uniquement **K points** autour d'une position de référence p_q .
- Ces points sont appris, comme les offsets dans une convolution déformable.
- On réduit ainsi les coûts de calcul et facilite l'entraînement.

Module d'attention déformable



Principe de l'attention déformable (1/3)

- Soit z_q le vecteur requête (feature) associé à la position de référence p_q dans une carte.
- On ne va plus faire interagir z_q avec tous les x_k , mais avec un sous-ensemble déformé :

$$\text{points déformés} = \{p_q + \Delta p_{mqk}\}_{k=1}^K$$

- Pour chaque tête m et chaque point k , on apprend :
 - un offset $\Delta p_{mqk} \in \mathbb{R}^2$
 - un poids d'attention $A_{mqk} \in [0, 1]$

Principe de l'attention déformable (2/3)

- La sortie de l'attention déformable pour la tête m est :

$$\text{Sortie}_m(z_q, p_q) = W_m \left[\sum_{k=1}^K A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$

- On applique une **interpolation bilinéaire** car les positions $p_q + \Delta p_{mqk}$ ne sont pas forcément entières.
- Les poids A_{mqk} sont normalisés via un softmax : $\sum_k A_{mqk} = 1$.

- Pour chaque requête (z_q, p_q) , on répète ce processus sur M têtes indépendantes :

$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M W_m \left[\sum_{k=1}^K A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$

- Où :
 - W'_m est une projection linéaire des features extraits.
 - W_m est une projection de sortie propre à la tête m .

- Comme une convolution déformable :
 - Offsets appris **par position** pour échantillonner des points pertinents.
 - Réutilise l'idée d'interpolation bilinéaire.
- Différences :
 - Ici, les offsets sont **dépendants du contenu** z_q (pas seulement de la position).
 - Chaque point est aussi pondéré par une attention **adaptive**.
- Le tout est différentiable et appris par descente de gradient.

Pourquoi c'est puissant ?

- Contrairement à l'attention dense classique :
 - L'attention déformable est **localisée mais flexible**.
 - Moins coûteuse **en calcul et mémoire**.
 - Meilleure convergence pendant l'entraînement.
- Chaque région peut accéder à une vue **guidée par le contenu** sur toute l'image, sans regarder partout.
- C'est ce qui permet aux modèles comme Mask2Former d'être rapides, précis, et efficaces en mémoire.

Attention Déformable Multi-Échelle (1/3)

- La plupart des modèles de détection modernes exploitent des cartes de caractéristiques multi-échelles.
- L'attention déformable peut naturellement s'étendre à cette configuration.
- Idée clé : pour chaque requête, on échantillonne des points non pas d'un seul niveau de caractéristique, mais de plusieurs niveaux (L niveaux).
- Cela permet à chaque requête de capter des indices à différentes résolutions.

Attention Déformable Multi-Échelle (2/3)

Formulation Générale

Entrée :

- $x^l \in \mathbb{R}^{C \times H_l \times W_l}$, pour $l = 1, \dots, L$ (niveaux multi-échelles)
- $p_q \in [0, 1]^2$: position normalisée de la requête

Sortie de l'attention multi-échelle :

$$\text{MSDeformAttn}(z_q, p_q, x_{l=1}^L) = \sum_{m=1}^M W_m \left[\sum_{l=1}^L \sum_{k=1}^K A_{mlqk} \cdot W'_m x^l (S_l(p_q) + \Delta p_{mlqk}) \right]$$

- $S_l(p_q)$: transformation de p_q vers la résolution du niveau l .
- Δp_{mlqk} : décalage relatif appris pour la tête m , niveau l , point k .
- A_{mlqk} : poids d'attention normalisé ($\sum_{l,k} A_{mlqk} = 1$).

Attention Déformable Multi-Échelle (3/3)

- Chaque requête est comme un drone qui peut regarder à plusieurs hauteurs (résolutions) et dans plusieurs directions (offsets), selon des priorités apprises (poids A_{mlqk}).
- Ce mécanisme capte à la fois les détails fins (niveau haut) et le contexte global (niveau bas).
- Si $L = 1$, $K = 1$, et $W_m = I$, le module devient une **convolution déformable** classique.
- Si on laisse les points de sampling explorer tout l'espace, cela revient à une **attention Transformer** classique (cas extrême).

Pixel Decoder dans Mask2Former

Contextualisation

- Dans Mask2Former, le **pixel decoder** transforme les cartes de caractéristiques en une représentation par pixel.
- Il utilise 6 couches de **Multi-Scale Deformable Attention** (MSDeformAttn).
- Résolutions d'entrée utilisées : $\frac{1}{8}$, $\frac{1}{16}$, $\frac{1}{32}$.
- L'objectif est de générer une carte de caractéristiques finale à résolution $\frac{1}{4}$.

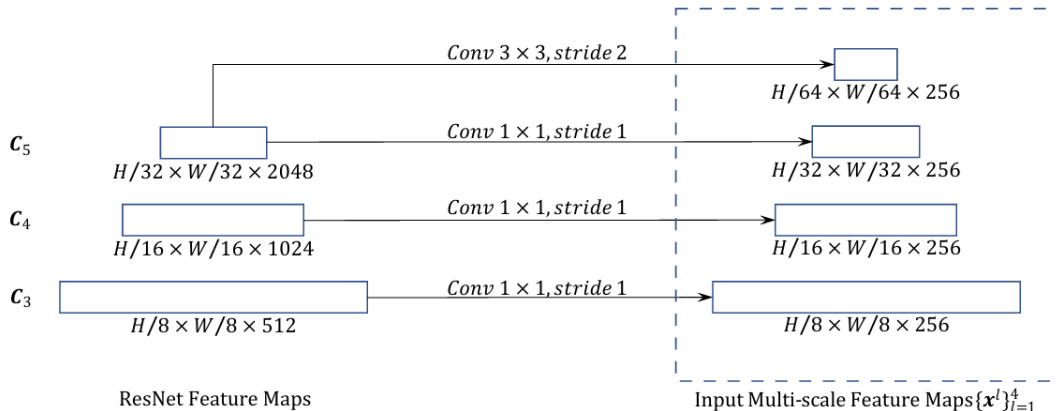
- Pour une requête (z_q, p_q) (contenu et position), le pixel decoder extrait un embedding riche :

$$\text{PixelFeat}_q = \text{MSDeformAttn}(z_q, p_q, x^{(1)}, x^{(2)}, x^{(3)})$$

- Chaque requête interroge plusieurs niveaux de features via un mécanisme d'attention sur des points échantillonnés dynamiquement.
- Cette opération remplace la fusion pyramidale classique des FPN.

Remarque : Cela permet d'encoder finement le contexte local et global dès le début du pipeline.

Construction de cartes de caractéristiques multi-échelles



Pixel Decoder : Reconstruction en Résolution Fine

- À partir des sorties MSDeformAttn, le pixel decoder reconstruit une feature map dense :

$$y(x, y) = \text{Conv}_{3 \times 3} \left(\text{Conv}_{1 \times 1}(F_l(x, y)) + \sum_{i,j} w_{ij}^{(x', y')} \cdot F_h(x_{ij}, y_{ij}) \right)$$

- $\Rightarrow$ Interpolation bilinéaire + addition avec une projection latérale, puis convolution de raffinement.
- Cette sortie est à résolution $\frac{1}{4}$ de l'image d'entrée.
- Elle est utilisée comme entrée pour le **transformer decoder**.

But : produire un embedding par pixel, prêt à être segmenté via attention croisée.

Bilan du Pixel Decoder

Ce que nous avons obtenu jusqu'ici

- Grâce aux 6 couches de **Multi-Scale Deformable Attention**, le pixel decoder a :
 - Fusionné efficacement les cartes multi-échelles $(\frac{1}{8}, \frac{1}{16}, \frac{1}{32})$.
 - Produit une carte de caractéristiques à résolution $\frac{1}{4}$.
 - Encodé chaque pixel avec un vecteur riche en contexte local et global.
- Cette carte de caractéristiques est appelée **Pixel Embedding Map**.
- Elle sert désormais de base pour une analyse sémantique plus fine.

Problème : *Comment convertir ces embeddings par pixel en segments d'objets significatifs ?*

Vers le Transformer Decoder

Pourquoi un Transformer Decoder ?

- Le pixel decoder fournit un **langage des pixels**, mais il manque encore une capacité à :
 - Capturer des relations entre régions (objets, instances...).
 - Regrouper les pixels en entités cohérentes.
- Le **Transformer Decoder** entre alors en scène pour :
 - Interagir avec les embeddings de pixels.
 - Générer des **masques** pour les objets ou segments cibles.
- C'est une étape essentielle pour aller de l'encoding spatial à une prédiction explicite de segments.

Problématique de l'ordre et de la position

Pourquoi a-t-on besoin d'un encodage positionnel ?

- Les modèles **transformers** ne possèdent pas de notion implicite de position : ils traitent les vecteurs d'entrée de manière **parallèle**.
- Contrairement aux réseaux convolutifs ou récurrents, ils n'ont **aucune structure spatiale** ou séquentielle « par défaut ».
- Dans une image, chaque pixel ou vecteur de feature est traité **sans savoir où il se trouve** (coin supérieur gauche ? centre ? bord droit ?).
- Cela empêche le modèle de :
 - Reconnaître des formes globales.
 - Capturer des relations spatiales (gauche/droite, haut/bas).
- **Solution** : injecter explicitement la position dans les embeddings via un **encodage positionnel** (sinusoïdal dans notre cas).

Encodage positionnel sinusoïdal

- Les Transformers ne connaissent pas l'ordre des éléments : il faut leur fournir la position de chaque entrée.
- L'encodage positionnel ajoute cette information aux embeddings.
- On utilise ici un encodage **sinusoïdal** proposé par Vaswani et al. (2017).
- Pour chaque position pos et dimension i .
- Chaque dimension encode une **fréquence différente**, ce qui permet de capturer des relations d'ordre et de distance.

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{\frac{2i}{d}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{\frac{2i}{d}}} \right)$$

Pourquoi la base 10000 ?

- **Spectre de fréquences contrôlé :**

- La formule $10000^{\frac{2i}{d}}$ génère des fréquences sur une échelle logarithmique.
- Permet de capturer des motifs locaux (hautes fréquences) et globaux (basses fréquences).

- **Stabilité de l'apprentissage :**

- Dérivées sin/cos bien comportées pour le gradient.
- Moins de risques de gradient explosif ou nul.

- **Généralisation naturelle :**

- Périodicité $\Rightarrow$ extrapolation possible à des positions jamais vues.
- Contrairement aux embeddings appris, pas de limite fixée à l'avance.

Exemple d'Encodage Positionnel : Étape 1 et 2

- Considérons une séquence de 3 mots : ["The", "cat", "sat"], avec dimension $d=4$.
- **Étape 1 : Embeddings tokens** (pré-calculés) :

$$E_{\text{The}} = [0.1, 0.2, 0.3, 0.4], \quad E_{\text{cat}} = [0.5, 0.6, 0.7, 0.8], \quad E_{\text{sat}} = [0.9, 1.0, 1.1, 1.2]$$

- **Étape 2 : Embeddings positionnels** calculés via sinus/cosinus pour chaque position :

$$PE_0 = [\sin(0), \cos(0), \sin(0), \cos(0)] = [0, 1, 0, 1]$$

$$PE_1 = [\sin(1), \cos(1), \sin(0.02), \cos(0.02)] \approx [0.8415, 0.5403, 0.01, 0.99995]$$

$$PE_2 = [\sin(2), \cos(2), \sin(0.04), \cos(0.04)] \approx [0.9093, -0.4161, 0.02, 0.9998]$$

Exemple d'Encodage Positionnel : Étape 3

- Étape 3 : Addition élément-wise des embeddings token et positionnels

$$E'_{\text{The}} = [0.1, 0.2, 0.3, 0.4] + [0, 1, 0, 1] = [0.1, 1.2, 0.3, 1.4]$$

$$E'_{\text{cat}} = [0.5, 0.6, 0.7, 0.8] + [0.8415, 0.5403, 0.01, 0.99995] = [1.3415, 1.1403, 0.71, 1.79995]$$

$$E'_{\text{sat}} = [0.9, 1.0, 1.1, 1.2] + [0.9093, -0.4161, 0.02, 0.9998] = [1.8093, 0.5839, 1.12, 2.1998]$$

- Ces embeddings combinés contiennent à la fois :
 - L'information **lexicale** (mot).
 - L'information **positionnelle** (ordre dans la séquence).

Pourquoi un encodage positionnel ?

- Le **pixel decoder** produit une carte de caractéristiques spatiales $x \in \mathbb{R}^{B \times C \times H \times W}$, où :
 - B = taille du batch,
 - C = nombre de canaux (features),
 - H, W = hauteur et largeur spatiale.
- Le Transformer traite chaque position indépendamment et en parallèle, il ne connaît donc pas la position spatiale intrinsèque des pixels.
- Pour que le Transformer puisse exploiter la structure spatiale, on ajoute un **encodage positionnel** qui informe chaque pixel de sa position (y, x) .

Le rôle du masque (mask) dans le Pixel Decoder

- Le **mask** est un tenseur booléen indiquant les pixels à **ignorer** : padding, zones hors cadre, ou régions non valides.
- Dans le pipeline, il sert à :
 - Ignorer ces pixels lors du calcul des positions spatiales cumulées pour l'encodage positionnel.
 - Garantir que les coordonnées spatiales sont continues et cohérentes uniquement sur les pixels valides.
 - Éviter que le Transformer porte attention à des pixels inutiles ou masqués.
- Dans le code, un masque initial vide (tout False) est souvent créé quand aucune zone invalide n'est présente, sinon il peut provenir d'étapes antérieures (padding, annotations).
- Ce mécanisme améliore la robustesse de l'encodage positionnel et des calculs d'attention dans les architectures complexes.

Masque et extraction des coordonnées spatiales

- Le **masque** $mask \in \{0, 1\}^{B \times H \times W}$ indique les pixels **invalides** (ex. zones à ignorer, bordures, padding).
- On définit $not_mask = 1 - mask$ pour repérer les pixels valides.
- Pour chaque pixel valide, on calcule ses coordonnées spatiales cumulées :

$$y_{embed}(b, y, x) = \sum_{i=0}^y not_mask(b, i, x), \quad x_{embed}(b, y, x) = \sum_{j=0}^x not_mask(b, y, j)$$

- Ces sommes donnent des indices croissants qui correspondent à la position verticale et horizontale réelles dans l'image.
- Si tous les pixels sont valides, ces coordonnées sont simplement les indices discrets (y, x) .

- Pour que les coordonnées restent dans une échelle stable, on peut **normaliser** ces valeurs :

$$y_{embed} \leftarrow \frac{y_{embed}}{y_{max} + \varepsilon} \times scale, \quad x_{embed} \leftarrow \frac{x_{embed}}{x_{max} + \varepsilon} \times scale$$

où y_{max} et x_{max} sont les valeurs maximales cumulées pour chaque batch,

$$scale = 2\pi$$

est choisi pour correspondre à la période des fonctions trigonométriques.

- Cette étape permet une uniformisation des positions entre différentes images ou tailles.

Encodage sinusoïdal multi-fréquences

- Le vecteur des fréquences est construit pour permettre une représentation multi-échelle :

$$dim_t[i] = temperature^{\frac{2\lfloor i/2 \rfloor}{d_{pos}}}$$

avec $temperature = 10000$ et d_{pos} le nombre de dimensions pour l'encodage positionnel.

- Pour chaque position spatiale, on divise les coordonnées normalisées par ces fréquences :

$$pos_x(b, y, x, i) = \frac{x_{embed}(b, y, x)}{dim_t[i]}, \quad pos_y(b, y, x, i) = \frac{y_{embed}(b, y, x)}{dim_t[i]}$$

- Les vecteurs positionnels sont calculés en alternant sinus et cosinus selon l'indice dimensionnel :

$$PE(2i) = \sin(pos(2i)), \quad PE(2i + 1) = \cos(pos(2i + 1))$$

- Ceci permet au modèle de capter différentes échelles spatiales et distances relatives.

Encodage positionnel : sortie et intégration

- Les encodages calculés pour x et y sont concaténés, formant un vecteur d'embedding positionnel de taille $2d_{pos}$ par pixel.
- Le tenseur retourné a la forme :

$$pos \in \mathbb{R}^{B \times 2d_{pos} \times H \times W}$$

- Il s'ajoute élément-wise aux features du pixel decoder avant d'être envoyés au Transformer Decoder.
- Cela permet au Transformer de discriminer la position spatiale des pixels tout en traitant les tokens en parallèle.
- **Résumé mathématique :**

$$z_{b,y,x} = f_{b,y,x} + PE_{b,y,x}$$

avec $f_{b,y,x}$ la feature pixel, et $PE_{b,y,x}$ l'encodage positionnel sinusoïdal.

Encodage de niveau (scale-level embedding)

- Chaque niveau de résolution (1/32, 1/16, 1/8) du pixel decoder a sa propre identité.
- Pour cela, on ajoute un **vecteur d'encodage spécifique à l'échelle** :

$$e_l^{lv} \in \mathbb{R}^{1 \times C}$$

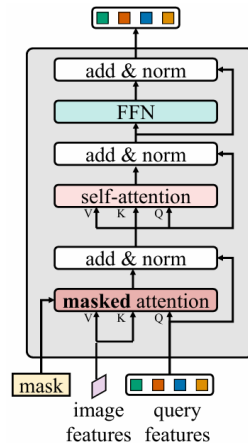
- C'est un vecteur **appris** durant l'entraînement, différent pour chaque échelle l .
- Il est ajouté à tous les pixels de la feature map du niveau l , indépendamment de leur position :

$$z_{b,y,x}^{(l)} = f_{b,y,x}^{(l)} + PE_{b,y,x}^{(l)} + e_l^{lv}$$

- Ceci permet au Transformer de distinguer les niveaux de résolution, même si des pixels ont les mêmes coordonnées relatives.

Introduction au Transformer Decoder

- Utilisation d'une **attention masquée** pour extraire des caractéristiques localisées.
 - L'attention croisée est restreinte à la région de premier plan prédite par le masque de chaque requête.
 - Évite l'attention sur toute la carte de caractéristiques.
- **Stratégie multi-échelle efficace** pour traiter les petits objets.
 - Utilise des cartes de caractéristiques de haute résolution.
 - Les différentes couches du décodeur reçoivent des niveaux de la pyramide de caractéristiques de façon circulaire.



Pourquoi une attention masquée ?

- L'attention croisée globale est coûteuse à apprendre :
 - Les modèles mettent du temps à apprendre à localiser les objets.
- **Hypothèse** : les régions locales suffisent pour mettre à jour les requêtes.
- Le contexte global peut être appris via **self-attention**.
- Solution proposée : **masked cross-attention**, concentrée sur la région de premier plan prédite par le masque.

attention_mask_intro.png

Définition du cross-attention standard

- À la couche l , l'attention croisée est définie par :

$$X^{(l)} = \text{softmax}(Q^{(l)}K^{(l)\top})V^{(l)} + X^{(l-1)}$$

- **Objectif** : mettre à jour les requêtes $X^{(l-1)}$ à l'aide des caractéristiques de l'image.
- L'ajout résiduel $+X^{(l-1)}$ stabilise l'apprentissage (residual learning).

- Définition :

$$Q^{(l)} = f_Q(X^{(l-1)}) \in \mathbb{R}^{N \times C}$$

- $X^{(l-1)}$: les requêtes en entrée de la couche l , avec N requêtes et C canaux.
- $f_Q : \mathbb{R}^{N \times C} \rightarrow \mathbb{R}^{N \times C}$ est une transformation linéaire.
- Chaque ligne $q_i \in \mathbb{R}^C$ représente une requête correspondant à un objet.

- Définition :

$$K^{(l)} = f_K(F^{(l)}), \quad V^{(l)} = f_V(F^{(l)}), \quad F^{(l)} \in \mathbb{R}^{H_l \times W_l \times C}$$

- $F^{(l)}$: carte de caractéristiques de l'image à l'échelle l
- On vectorise : $K^{(l)}, V^{(l)} \in \mathbb{R}^{S \times C}$ avec $S = H_l W_l$
- Chaque position (x, y) de l'image devient une clé k_{xy} et une valeur v_{xy}

Attention masquée : introduction du masque

- Formule modifiée :

$$X^{(l)} = \text{softmax}(M^{(l-1)} + Q^{(l)}K^{(l)\top})V^{(l)} + X^{(l-1)}$$

- $M^{(l-1)} \in \mathbb{R}^{N \times S}$ est un masque additif :

$$M^{(l-1)}(i, j) = \begin{cases} 0 & \text{si la position } j \text{ est dans le masque de } q_i \\ -\infty & \text{sinon} \end{cases}$$

- Le softmax devient localisé : seules les positions autorisées sont prises en compte.

Self-Attention : Formulation mathématique

- Soit $X^{(l)} \in \mathbb{R}^{N \times C}$ la sortie du masked cross-attention.
- On applique une opération de self-attention sur ces N queries :

$$\tilde{X}^{(l)} = \text{softmax} \left(\frac{Q_s^{(l)} K_s^{(l)\top}}{\sqrt{d_k}} \right) V_s^{(l)} + X^{(l)}$$

- Avec :

$$Q_s^{(l)} = X^{(l)} W_Q \in \mathbb{R}^{N \times d_k}$$

$$K_s^{(l)} = X^{(l)} W_K \in \mathbb{R}^{N \times d_k}$$

$$V_s^{(l)} = X^{(l)} W_V \in \mathbb{R}^{N \times d_v}$$

- W_Q , W_K , W_V sont des matrices de projection apprises.

- On obtient un tenseur $\tilde{X}^{(l)} \in \mathbb{R}^{N \times C}$, raffiné par auto-attention.
- Ces features servent ensuite (après normalisation) à alimenter le bloc suivant.
- Ce processus améliore la cohérence entre régions masquées proposées.
- **Important** : Pas de signal image direct ici, uniquement interactions entre queries.

Pourquoi un réseau Feed-Forward après l'attention ?

- L'attention (cross ou self) permet aux queries d'échanger des informations entre elles.
- Mais ces interactions sont **linéaires** : on combine des vecteurs, on ne les transforme pas encore profondément.
- On a besoin d'un **réseau non linéaire** (MLP) pour transformer chaque query individuellement :
 - amplifier certaines dimensions,
 - introduire des interactions plus complexes entre les composantes du vecteur.
- L'attention est une discussion collective, le FFN est une **prise de décision personnelle** de chaque query après réflexion.

Feed-Forward Network (FFN) : Formulation mathématique

- À la sortie de l'auto-attention, on applique un réseau feed-forward identique à chaque query :

$$Z^{(l)} = \text{FFN}(\tilde{X}^{(l)}) = \text{ReLU} \left(\tilde{X}^{(l)} W_1 + b_1 \right) W_2 + b_2$$

- Avec :

- $W_1 \in \mathbb{R}^{C \times d_{\text{ff}}}$, $b_1 \in \mathbb{R}^{d_{\text{ff}}}$
- $W_2 \in \mathbb{R}^{d_{\text{ff}} \times C}$, $b_2 \in \mathbb{R}^C$

- Ce MLP est appliqué indépendamment à chaque ligne de $\tilde{X}^{(l)} \in \mathbb{R}^{N \times C}$

- Le FFN agit comme un **filtre non linéaire** : chaque query est enrichie indépendamment.
- La structure W_1 puis W_2 permet :
 - une projection dans un espace plus grand ($d_{\text{ff}} > C$) pour exprimer plus de variations,
 - puis un retour à la dimension initiale avec W_2 .
- C'est comme envoyer chaque vecteur dans un laboratoire pour traitement individuel et retour.
- Cela permet au modèle de raffiner les informations internes de chaque objet détecté.

- Chaque vecteur $z_i \in \mathbb{R}^C$ (après FFN) produit deux prédictions :

① **Logits de classe** via projection linéaire :

$$\hat{y}_i = z_i W_{\text{cls}} + b_{\text{cls}} \in \mathbb{R}^{K+1}$$

(classe la plus probable par softmax)

② **Masque d'objet** via une tête de masque (fusion avec le backbone) :

$$\hat{m}_i = f_{\text{masque}}(z_i, \text{features}) \in [0, 1]^{H \times W}$$

- Ces deux prédictions sont ensuite combinées pour générer une segmentation complète.

Fusion vecteur-image pour la prédiction des masques

- Chaque vecteur $z_i \in \mathbb{R}^C$ encode une entité (ex : *villa, appartement, route...*).
- Le Pixel Decoder fournit une carte dense $F_{\text{pixel}} \in \mathbb{R}^{C \times H \times W}$ contenant l'information spatiale multi-échelle.
- On projette z_i dans le même espace que F_{pixel} :

$$p_i = W_{\text{mask}} z_i \in \mathbb{R}^C$$

- À chaque pixel (h, w) , on calcule un produit scalaire :

$$\hat{m}_i[h, w] = \langle p_i, F_{\text{pixel}}[:, h, w] \rangle$$

- Une fonction sigmoïde est appliquée pour obtenir la probabilité d'appartenance au masque :

$$\hat{m}_i = \sigma \left(p_i^\top F_{\text{pixel}} \right)$$

Résumé : Pipeline de prédiction dans Mask2Former

① Backbone CNN :

- Extrait des features multi-échelles de l'image : $\{F_1, F_2, \dots, F_4\}$

② Pixel Decoder (MSDeformAttn) :

- Combine ces features en une carte haute résolution F_{pixel}

③ Transformer Decoder :

- À chaque étape l , il raffine des queries $z_i^{(l)}$ via attention croisée + self-attention + FFN.
- Produit N vecteurs finaux z_i

④ Têtes de prédiction :

- **Logits de classe** : $\hat{y}_i = \text{MLP}_{\text{cls}}(z_i)$
- **Masques** : $\hat{m}_i = \sigma(W_{\text{mask}} z_i \cdot F_{\text{pixel}})$

Fonction de perte dans Mask2Former : Vue d'ensemble

Objectif : Apprendre à chaque prédiction $(\hat{y}_i, \hat{m}_i)$ à correspondre à un objet réel (classe y_j , masque m_j).

- Chaque prédiction est évaluée avec une combinaison de deux pertes :

$$\mathcal{L}_{\text{mask}} = \lambda_{\text{ce}} \cdot \mathcal{L}_{\text{ce}} + \lambda_{\text{dice}} \cdot \mathcal{L}_{\text{dice}}$$

$$\mathcal{L}_{\text{finale}} = \mathcal{L}_{\text{mask}} + \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}}$$

- **Trois composants** :

- $\mathcal{L}_{\text{ce}}$: Cross-entropy sur les masques binaires.
- $\mathcal{L}_{\text{dice}}$: Mesure la similarité globale entre masques prédits et masques cibles.
- $\mathcal{L}_{\text{cls}}$: Cross-entropy pour la classe prédite.

- Poids utilisés dans l'article :

$$\lambda_{\text{ce}} = 50, \quad \lambda_{\text{dice}} = 50, \quad \lambda_{\text{cls}} = 20$$

Rôle de l'algorithme hongrois dans la perte

Problème : Chaque image contient plusieurs objets et Mask2Former produit N prédictions. Qui correspond à quoi ?

Solution : L'algorithme **hongrois** résout un problème d'assignation optimale.

- Métaphore : les prédictions sont des *candidats*, les objets réels sont des *postes à pourvoir*.
- Le recruteur (algorithme) évalue chaque correspondance selon une *coût total* :

$$\text{coût}(i, j) = \mathcal{L}_{\text{cls}}(i, j) + \mathcal{L}_{\text{mask}}(i, j)$$

- Le matching optimal minimise la somme totale des coûts.
- Seules les paires appariées contribuent à la fonction de perte totale.
- Les prédictions non appariées reçoivent une perte pour "objet absent" (classe fond).

Intuition : On veut que chaque prédiction trouve "le bon objet" à imiter, et que les "mauvais" candidats ne soient pas encouragés.

Perte *Cross-Entropy* sur les masques (1/2)

Contexte : Chaque prédiction $\hat{m}_i \in [0, 1]^{H \times W}$ doit approcher un masque binaire $m_j \in \{0, 1\}^{H \times W}$ associé à un objet réel.

Définition : la perte *pixel-wise binary cross-entropy* entre $\hat{m}_i$ et m_j est donnée par :

$$\mathcal{L}_{ce}(i, j) = -\frac{1}{HW} \sum_{x=1}^H \sum_{y=1}^W [m_j(x, y) \log \hat{m}_i(x, y) + (1 - m_j(x, y)) \log(1 - \hat{m}_i(x, y))]$$

Note : $\hat{m}_i(x, y) = \sigma((W_{\text{mask}} z_i)^\top F_{\text{pixel}}(x, y))$ où σ est la sigmoid.

Remarques importantes :

- La BCE est calculée pour chaque paire (i, j) prédiction / masque GT.
- Les masques sont calculés à partir de la multiplication point-wise :

$$\hat{m}_i(x, y) = \sigma \left(\sum_{c=1}^C w_c^{(i)} \cdot F_{\text{pixel}, c}(x, y) \right)$$

où $w^{(i)} = W_{\text{mask}} z_i$ est un filtre 1×1 dépendant du query.

- $\mathcal{L}_{\text{ce}}$ est différentiable, pixel-wise, mais sensible au déséquilibre (foreground / background).

Perte *Dice* pour les masques (1/2)

But : Comparer globalement les zones d'intersection entre prédiction $\hat{m}_i$ et masque réel m_j .

Définition continue :

$$\mathcal{L}_{\text{dice}}(i, j) = 1 - \frac{2 \sum_{x,y} \hat{m}_i(x, y) \cdot m_j(x, y)}{\sum_{x,y} \hat{m}_i(x, y)^2 + \sum_{x,y} m_j(x, y)^2 + \varepsilon}$$

où $\varepsilon > 0$ est un terme de régularisation pour éviter la division par zéro.

Note : $\hat{m}_i(x, y) \in [0, 1]$ est une prédiction soft, $m_j(x, y) \in \{0, 1\}$ est un masque binaire.

Propriétés :

- La Dice loss est une approximation continue du Dice coefficient :

$$\text{Dice}(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

- Invariante aux déséquilibres de classes (foreground vs background).
- Complémentaire à la cross-entropy pour une supervision plus robuste.

Application : utilisée conjointement avec $\mathcal{L}_{\text{ce}}$:

$$\mathcal{L}_{\text{mask}}(i, j) = \lambda_{\text{ce}} \cdot \mathcal{L}_{\text{ce}}(i, j) + \lambda_{\text{dice}} \cdot \mathcal{L}_{\text{dice}}(i, j)$$

Assignment optimale par l'algorithme hongrois (1/2)

Problème : On prédit N couples $(\hat{y}_i, \hat{m}_i)$ mais seulement M objets annotés (y_j, m_j) .

- On construit une matrice de coût $C \in \mathbb{R}^{N \times M}$:

$$C(i, j) = \mathcal{L}_{\text{cls}}(\hat{y}_i, y_j) + \mathcal{L}_{\text{mask}}(\hat{m}_i, m_j)$$

- L'assignation $\sigma^* \in \mathfrak{S}_M$ (permutation de M indices parmi N) minimise le coût total :

$$\sigma^* = \arg \min_{\sigma \in \mathfrak{S}_M} \sum_{j=1}^M C(\sigma(j), j)$$

Déroulement :

- ➊ Pour chaque image : construire $C(i, j)$ pour toutes les paires prédiction / vérité terrain.
- ➋ Utiliser l'algorithme hongrois pour trouver σ^* minimisant le coût total.
- ➌ Appliquer les fonctions de perte uniquement sur les paires appariées $(\hat{y}_{\sigma^*(j)}, \hat{m}_{\sigma^*(j)})$.
- ➍ Les prédictions non appariées sont supervisées avec la classe "no-object".

Avantage : formulation *set-based*, sans post-traitement ni NMS, directement optimisable.

Définitions (pour une image donnée) :

- N : nombre de prédictions produites par le modèle.
- M : nombre d'objets annotés dans l'image (masques + labels).
- $\hat{y}_i \in \mathbb{R}^{K+1}$: logits de classe produits pour la prédiction i (K classes + 1 classe spéciale "no-object").
- $\hat{m}_i \in [0, 1]^{H \times W}$: prédiction de masque soft.
- (y_j, m_j) : label et masque binaire du j -ème objet réel.
- $\sigma^* \in \mathfrak{S}_M$: permutation optimale définie par l'algorithme hongrois (voir slides précédentes).

Objectif : Superviser les logits $\hat{y}_i$ associés à chaque prédiction i .

Définition (Cross-Entropy sur softmax) : pour une prédiction i appariée à un objet de classe $y_j \in \{1, \dots, K\}$,

$$\mathcal{L}_{\text{cls}}(i, j) = -\log \left(\frac{e^{\hat{y}_i^{(y_j)}}}{\sum_{k=0}^K e^{\hat{y}_i^{(k)}}} \right)$$

où :

- $\hat{y}_i^{(k)}$: logit pour la classe k .
- $k = 0$ est la classe spéciale “no-object”.
- $\hat{y}_i$ passe par une softmax sur $K + 1$ classes.

Traitement des prédictions non appariées ("no-object")

Contexte : N prédictions produites mais seulement M appariées via σ^* .

Définition de l'ensemble non apparié :

$$U = \{i \in \{1, \dots, N\} \mid i \notin \text{Im}(\sigma^*)\}$$

Supervision : on applique une cross-entropy avec la classe 0 :

$$\mathcal{L}_{\text{cls}}(i, 0) = -\log \left(\frac{e^{\hat{y}_i^{(0)}}}{\sum_{k=0}^K e^{\hat{y}_i^{(k)}}} \right) \quad \text{pour } i \in U$$

Note : ces prédictions ne reçoivent aucune perte de masque.

Formulation complète de la fonction de perte

Pertes sur les prédictions appariées :

$$\sum_{j=1}^M [\mathcal{L}_{\text{cls}}(\sigma^*(j), j) + \lambda_{\text{ce}} \cdot \mathcal{L}_{\text{ce}}(\sigma^*(j), j) + \lambda_{\text{dice}} \cdot \mathcal{L}_{\text{dice}}(\sigma^*(j), j)]$$

Pertes sur les prédictions non appariées :

$$\sum_{i \in U} \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}}(i, 0)$$

Perte totale (pour une image) :

$$\mathcal{L}_{\text{total}} = \sum_{j=1}^M [\mathcal{L}_{\text{cls}}(\sigma^*(j), j) + \mathcal{L}_{\text{mask}}(\sigma^*(j), j)] + \sum_{i \in U} \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}}(i, 0)$$

Synthèse : Fonction de perte dans Mask2Former

Composants :

- $\mathcal{L}_{ce}$: Cross-entropy binaire pixel-wise sur les masques.
- $\mathcal{L}_{dice}$: Dice loss pour la forme globale des masques.
- $\mathcal{L}_{cls}$: Cross-entropy multi-classes sur les logits de classe.

Matching :

- L'algorithme hongrois sélectionne l'appariement optimal selon :

$$\text{coût}(i, j) = \mathcal{L}_{cls}(i, j) + \mathcal{L}_{mask}(i, j)$$

Pertes finales :

$$\mathcal{L}_{mask} = \lambda_{ce} \cdot \mathcal{L}_{ce} + \lambda_{dice} \cdot \mathcal{L}_{dice}, \quad \mathcal{L}_{total} = \mathcal{L}_{mask} + \lambda_{cls} \cdot \mathcal{L}_{cls}$$

Objectif : évaluer à la fois la précision des **masques** et la **classification** des instances dans une image.

Métriques principales :

- **PQ** : *Panoptic Quality*, métrique principale pour la segmentation panoptique.
- **PQ_{Th}**, **PQ_{St}** : PQ restreinte aux "choses" (instances distinctes) et aux "trucs" (régions amorphes).
- **AP_{Th}** : *Average Precision* des objets instanciés (choses uniquement).
- **mIoU_{pan}** : Intersection over Union moyenne pondérée sur toutes les classes.
- **#params**, **FLOPs** : coût en complexité.
- **FP** : taux de fausses prédictions (instances absentes dans le GT).

Panoptic Quality (PQ)

Définition :

$$PQ = \frac{\sum_{(p,g) \in TP} \text{IoU}(p, g)}{|TP|} \cdot \frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}$$

Détails :

- (p, g) : paire prédiction / vérité terrain ayant $\text{IoU} > 0.5$.
- TP : prédictions correctement appariées.
- FP : prédictions sans correspondance ($\text{IoU} < 0.5$ avec tout GT).
- FN : objets GT non détectés par le modèle.

Décomposition :

$$PQ = SQ \cdot RQ \quad \text{où} \quad SQ = \text{quality moyenne (IoU)}, \quad RQ = \text{rappel pondéré}$$

Deux sous-métriques de PQ :

- PQ_{Th} : PQ calculé uniquement sur les **choses** (objets instanciés : personne, voiture, etc.).
- PQ_{St} : PQ calculé uniquement sur les **trucs** (régions amorphes : ciel, herbe, route, etc.).

Utilité : ces métriques permettent d'évaluer séparément :

- La capacité à détecter et segmenter des instances distinctes (PQ_{Th}).
- La qualité de la segmentation sémantique continue (PQ_{St}).

AP_{Th} : Average Precision pour les choses

Définition : Moyenne des précisions à différents seuils IoU (standard COCO) pour les objets instanciés (“choses”).

$$AP_{Th} = \frac{1}{T} \sum_{t \in \{0.5, 0.55, \dots, 0.95\}} AP(t)$$

où :

- $AP(t)$: précision au seuil d'IoU t .
- Moyenne calculée uniquement sur les classes de “choses”.

Note : AP est plus sensible à la précision du *bounding box + masque* que PQ.

Définition :

$$\text{mIoU}_{pan} = \frac{1}{C} \sum_{c=1}^C \frac{|P_c \cap G_c|}{|P_c \cup G_c|}$$

où :

- C : nombre total de classes (choses + trucs).
- P_c : ensemble de pixels prédits pour la classe c .
- G_c : ensemble de pixels au sol (GT) pour la classe c .

Utilité : métrique globale qui ignore l'instance, purement segmentation par classe.

Average Precision (AP) : Moyenne de la précision sur différents seuils d'IoU.

- **AP** : moyenne sur $\text{IoU} \in [0.5, 0.95]$ par pas de 0.05.
- **AP_S** : AP pour les objets de petite taille ($\text{area} < 32^2$ pixels).
- **AP_M** : objets moyens ($32^2 \leq \text{area} < 96^2$).
- **AP_L** : objets larges ($\text{area} \geq 96^2$).

$$\text{AP} = \frac{1}{T} \sum_{t=1}^T \text{AP}_{\text{IoU}_t}, \quad T = 10$$